
Twenty-Eight Ways to Measure Nothing: A Failure Catalogue from Instrumenting an Agent Harness

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 An OS layer for agentic AI will standardise checkpointing, recovery, and observ-
2 ability, and those designs can only be compared if they can be measured. We built
3 the instrument such comparison requires — an observational timeline committed
4 to git, exhaustive replay of held-out tests at every observation, detection attributed
5 by coverage — and we report two things.

6 First, the instrument’s failure record: **twenty-eight defects, of which twenty-three**
7 **produced a plausible number rather than an error, and all but one were present**
8 **while a green test suite watched.** They collapse into four mechanisms — ambient-
9 environment dependence, infrastructure conflated with measurement, final-state
10 measurement of event-shaped quantities, and output consumed without asserting
11 its producer — and we show each mechanism operating in public infrastructure,
12 including the official SWE-bench grader. The capstone: after nineteen fixes, every
13 validation gate we had passed, and the headline number was still wrong, because
14 the agent executed on the host while every check validated the measurement path.
15 A gate certifies only the paths it exercises.

16 Second, what the corrected instrument measures. On the same benchmark instances,
17 under the same harness and recovery policy, one frontier model stack produced
18 **zero regression events across 40 runs** while another produced **140, in 11 distinct**
19 **breakage incidents across 6 bearing runs** (equal-N Fisher $p = 0.011$) — every
20 recovered event erased by the harness’s own rollback and invisible in the final tree.
21 One officially-*resolved* patch carried three of them, two undetected even by the
22 harness that produced it. With the pre-declared recovery-policy contrast unblinded:
23 no recovery leaves more of this damage in the final tree than rollback (paired sign
24 $p = 0.022$) — and rollback resolves fewer tasks, because the verifier that gates it
25 rejects good patches. Regressions during agent runs are a property of the agent
26 regime, not the benchmark, and the final state — all existing evaluation examines
27 — is where the evidence is not.

1 Introduction

29 The agentic-systems community is converging on an OS layer: shared abstractions for memory,
30 scheduling, checkpointing, recovery. Choosing between abstractions requires evidence, and this
31 paper is about how much harder producing that evidence is than it looks — not in the statistics, but in
32 the plumbing that produces the numbers the statistics consume.

33 Our motivating question is narrow. When a step in a long-horizon agent run fails verification,
34 deployed harnesses either **repair in place** (hand the model its own broken tree) or **roll back and**
35 **retry** (reset to the last verified checkpoint). The second discards work and repays cold-cache token

36 prices; its defence is that it leaves less collateral damage. That claim is testable — if you can say, for
37 every run, *when* previously-working behaviour stopped working.

38 **Contributions.** (i) The instrument: observational checkpointing on a hidden git ref, exhaustive replay,
39 coverage-based attribution, and a validation regime whose controls include re-measuring archived
40 runs. (ii) A twenty-eight-defect catalogue with the mechanism for each, in a four-class taxonomy
41 whose classes we show operating in public harnesses. (iii) First measurements from the corrected
42 pipeline: across the same 40 instances, two frontier stacks at equal budgets give a 0-versus-140-event
43 contrast in which final-state evaluation sees one event in 184; and the pre-declared recovery-policy
44 contrast, unblinded with its committed analysis, shows rollback keeps final trees clean at a measurable
45 cost in resolution.

46 2 What has to be measured

47 A **regression event** is a held-out test that passed at some observation of a run and fails at a later one.
48 Three properties make it hard to measure honestly.

49 **It is an event, not a state.** A regression introduced and then repaired leaves no trace at the end of the
50 run — and repair is exactly what a recovery policy does. An instrument that inspects the final artifact,
51 which is how regressions on this benchmark are measured in published work [3], records zero by
52 construction for every recovered event. §5.1 measures how much that misses: in our data, all of it.

53 **Its absence and the instrument's death are the same observation.** Zero is what a clean run
54 produces — and what a dead probe, an unmatched parser, and an empty timeline produce. §4 is
55 twenty-eight instances of this.

56 **Detection must be attributed, not co-located.** "The harness failed something while the regression
57 was open" credits whichever policy fails most often with the best detection. Attribution needs a causal
58 join: a failing harness check and the broken held-out test must exercise a file the agent changed. We
59 report three levels — attributed detection, co-occurrence (an over-count of detection), and unknown
60 — and never let "could not measure" render as any of them.

61 3 The instrument, and how it is validated

62 **Observational checkpointing.** Every mutating tool call commits the tree to a git ref the agent cannot
63 enumerate, from a private index, so the agent's own `git status` and `git diff` are byte-identical
64 with the instrument on or off. Rollbacks and run-end are observed too — the recovering policy's
65 recoveries must be visible to the timeline that scores it.

66 **Exhaustive replay.** Held-out tests are replayed at *every* observation inside the instance's pinned
67 container. Bisection would assume monotone verdicts; non-monotonicity is the treatment (row C1).

68 **Attribution by coverage,** built once per instance at the base commit, identical across arms by
69 construction.

70 **Validation.** Five pre-set gates (negative control, positive control including *recovered* injections, flake
71 screen, unknown-rate ceiling, baseline liveness), plus the control the git substrate makes possible:
72 **re-scoring** — an archived run's committed timeline re-measured under a changed instrument, no
73 model calls, only the instrument varying. This control, exercised twice on different archived runs,
74 reproduced their episode counts exactly each time — once localising a disagreement to the runs rather
75 than the detector, once proving that forty fresh runs' zero (§5.1) belonged to the runs.

76 4 Twenty-eight ways to measure nothing

77 Every row is a real defect from this work. **S** = silent: it produced a plausible number rather than
78 an error. **G** = present while the project's test suite was green (row A4 *is* the suite, marked —). **H**
79 = findable only on a clean host, by construction. The census closes at submission; three further
80 defects caught by the §5.2 golden gate before they could produce a number are described there and
81 deliberately not counted here.

82 4.1 The catalogue, by mechanism

83 **Class A — ambient-environment dependence.** The code asks the machine a question and gets a
84 different answer on a different machine.

85

#	Defect	Consequence	S	G	H
A1	Shadow commits inherit the machine's git identity	timeline silently empty on any clean machine	✓	✓	✓
A2	Gate probe invoked bare python	every probe exit 127 on clean Ubuntu	✗	✓	✓
A3	Unpinned SDK resolved a different major version	two machines ran different code	✗	✓	✓
86 A4	Test fixtures verified with bare pytest from PATH	15 tests fail on a clean host, presenting as harness defects	✗	—	✓
A5	A benchmark scorer invoked bare python	score 0.0 manufactured by a missing interpreter	✓	✓	✓
A6	Agent executed on the host; measurement in the pinned image	the capstone — §5	✓	✓	✓

87 **Class B — infrastructure failure conflated with measurement.** The instrument's zero and the
88 defect's zero are the same number.

89

#	Defect	Consequence	S	G	H
B1	Output marker used shell : (prints nothing)	a perfect 13/13 run scored as 13 errors	✓	✓	
B2	Results on stderr, markers on stdout	one framework's results fell outside the parsed slice	✓	✓	
B3	Probe diffed against a deliberately-deleted commit	every graded test a hole → "0 regressions"	✓	✓	
B4	Interrupted mirror clone accepted with zero commits	later instances die far from the cause	✓	✓	
90 B5	Container workdir unvalidated	every exec exit 127, OCI error on stdout	✓	✓	
B6	Isolation removed loopback, not just the internet	23.5% of the oracle dead; one instance 812 tests	✓	✓	
B7	Provider cache served force-removed containers	later cells of an instance replay against a corpse and score clean	✓	✓	
B8	Per-observation tree reset reverted image build-time edits	a repo family's oracle dead, disguised as flake	✓	✓	

91 **Class C — final-state measurement of an event-shaped quantity.** The number measured is not the
92 quantity defined.

93

#	Defect	Consequence	S	G	H
C1	Bisection assumed monotone verdicts	recovered regressions invisible; recovering policy flattered	✓	✓	
C2	The declared event unit was implemented nowhere	pipeline counted per parametrised test; $\hat{\lambda}$ off by up to $2.7\times$	✓	✓	
94 C3	Rollbacks produced no observation	the recovering arm's recoveries invisible to the timeline scoring it	✓	✓	
C4	"Persists past the step boundary" defined nowhere	one legal reading excludes the events the design exists to count	✓	✓	

95 **Class D — output consumed without asserting its producer.**

96

#	Defect	Consequence	S	G	H
D1	Audit-trail field never populated	every checkpoint reported 0 tool errors, always	✓	✓	
D2	Malformed planner output crashed the run	33% of runs lost, scored as task failures	✗	✓	
D3	Dependency install counted as agent work	3,640 of 3,641 changed files; attribution vacuous	✓	✓	
D4	Join preferred the first observation over the last	failures pinned to the earliest tree under the fine grid	✓	✓	
D5	Executed config rebuilt from a name	manifest describes a run that never happened	✓	✓	
97 D6	Git handles never released	sweeps die of fd exhaustion after ~400 cells, loss concentrated on the back half	✓	✓	
D7	Console re-walked the full tree per run	answers first request, then presents as a dead server	✗	✓	
D8	Detection events never carried failing-test ids while attribution joined on them	attributed detection structurally UNKNOWN on every real run	✓	✓	
D9	Absent coverage rendered as measured silence	8 unmeasured episodes reported as attributed silence with unknown-rate 0.0	✓	✓	
D10	Turns that hit the output ceiling were executed anyway	truncated-but-parseable JSON wrote half a file; a 78-event storm read as agent incompetence, and the adapter remapped the stop reason that said why	✓	✓	

98 Totals, computed from the tables: **23 of 28 silent; 27 of 28 present under a green suite (the 28th**
99 **being the suite itself); 6 of 28 findable only on a clean host.** D9 and D10 earn a sentence of
100 humility: one appeared in the first valid run’s own sidecar after an audit predicted it; the other on first
101 contact with a second model family, whose longer outputs turned a latent ceiling into a mangled tree.
102 The class does not exhaust.

103 4.2 The two design rules that caught most of them

104 **Infrastructure failure is a missing observation, never a test failure** — a typed error, never fail.
105 This converted B1, B2, B3, B5 and B6 from fabricated regressions into visible holes.

106 **Zero events must be distinguishable from a dead instrument.** The baseline-liveness gate exists
107 because an instrument whose every probe errored once scored a perfect negative control; on first
108 contact with a clean host it failed loudly on A2 — the defect that would otherwise have been certified
109 as a clean result.

110 4.3 The working practice that found the rest

111 **Never consume a measurement’s output without first asserting the measurement succeeded.**
112 D6 hid for weeks because a fixture consumed a sweep’s output without checking the sweep’s status.
113 Nearly every row above is some component violating this rule; it is also how the official SWE-bench
114 grader could be induced to mark unresolved patches resolved by forged stdout [5].

115 5 The capstone: every gate passed, and the number was still wrong

116 Three early pilots on SWE-bench Verified (80 runs, \$99.07, development slice) measured an event
117 rate far below the pre-declared gate. The gates had been fixed in advance; re-scoring confirmed
118 detection had not regressed; we drafted the pre-declared conclusion — switch substrates.

119 The conclusion was wrong. The harness materialised each task as a bare checkout on the host and ran
120 the agent there, while every probe, verdict, and validation gate ran inside the task’s pinned image.
121 On the host, `import matplotlib` in that checkout *succeeds* — the working directory shadows the
122 installed package as an uncompiled namespace package — and everything downstream fails in ways
123 indistinguishable from agent incompetence. 26 of 28 zero-step runs traced here. We withdrew the
124 substrate claim: the pre-declared rule behaved correctly on the data it was given; the *inference* did
125 not survive the data’s provenance. No gate had exercised the agent’s execution path. **A validation**

126 **gate certifies only the paths it exercises**, and the fix is the architecture the instrument already used
 127 for its probes: the agent must execute where it is measured.

128 5.1 What the corrected pipeline measures

129 All numbers exploratory, on a pre-declared development slice, excluded from any confirmatory frame.
 130 The declared event unit is one (test function, onset observation) pair, parametrised variants collapsed;
 131 we report **incidents** (distinct onset observations), **events** (declared unit), and **bearing runs** together,
 132 because the data are overdispersed and no single rate summarises them honestly.

133 **Stack one — Claude planner/worker, 40 instances, rollback policy:** resolve 32.5% (13/40, exact
 134 CI [18.6%, 49.1%]), and **zero events in 227 exhaustively-replayed observations** (bearing 0/40, CI
 135 [0%, 8.8%]). The same regime’s earlier two-instance canary produced one bearing run (3 declared
 136 events), so the regime’s pooled rate is near zero, not literally zero. The zero is demonstrated to belong
 137 to the runs: re-scoring that canary’s archived timeline under this identical instrument reproduces its
 138 events exactly (§3).

139 **Stack two — GPT-5.6 planner/worker, the 10-instance prefix of the same slice, same harness,
 140 policy, and budgets:** resolve 5/10 at a fifth of the cost, and **57 events across 4 incidents, concen-
 141 trated in 2 of 10 runs** (bearing CI [2.5%, 55.6%]). Claude’s runs on the same 10 instances: 6/10
 142 resolve, zero events. At $n=10$ the run-level bearing contrast is not by itself significant (Fisher exact p
 143 ≈ 0.24 , 0/10 vs 2/10); the finding is the event-level record, and the equal- N forty-instance run and
 144 the recovery-policy contrast were executing, pre-declared, at submission time. Exposure does not
 145 explain the contrast: the stacks’ observation densities differ by $1.4\times$ (8.2 vs 5.7 per run) while the
 146 event totals differ by 57-to-0 — per observation, 0.70 versus 0.00.

147 Two disclosures. This calibration is a re-run: the first attempt was destroyed by defect D10 (a
 148 truncated write produced a 78-event storm; the sweep’s circuit breaker stopped it at \$1.12) and is
 149 excluded as infrastructure, with the guard’s revert-verified test as the evidence the re-run is sound —
 150 no capped turns appear in its event logs. And the pre-declared gate for proceeding on this substrate
 151 still *fails* under stack two — $\hat{\lambda}$ clears its threshold but bearing $20\% < 25\%$ — so what these numbers
 152 license is not a confirmatory pass but the regime-conditional re-declaration of the gates, filed as a
 153 pre-registration amendment before the contrast arms are unblinded.

154 **What the events were.** Every one of the 57 was erased by the harness’s own rollback before the
 155 run ended; every final tree grades clean (59/59 and 145/145 on the bearing instances). So all 57
 156 are invisible to final-state evaluation — the population prior work measures [3]. Within them, the
 157 harness’s own view splits three ways: 39 were *detected and erased* (a failing check attributable to the
 158 regression, then rollback); 16 more co-occurred with some failure without an attributable link; and —
 159 on the run that matters most — **an officially-resolved patch (all graded tests green) carried three
 160 regressions, two of which the harness never noticed at all.** One stack’s destructive edit broke 54
 161 test functions at a single observation and left a perfect final state; a leaderboard cannot distinguish
 162 that run from one that never broke anything.

163 The undercount, measured across every bearing run in both sweeps:

	event-level record	visible in the final state
164 8 bearing runs, both sweeps	184 declared events	1

165 Final-state measurement — the instrument all published regression auditing uses [3] — captures
 166 0.5% of what the timeline records here. The other 99.5% was created and repaired between the two
 167 states it examines.

168 5.2 The golden gate

169 The seam that closes A6 is validated by a *golden check*: the benchmark’s own gold patch driven
 170 through the real routed tool path at \$0 of model spend must grade resolved; a null run must not.
 171 Across the three hardest repo families it passed, catching three more defects before they could cost
 172 anything — one of which would have deflated every resolve rate ~ 3 points forever. On a rolling
 173 benchmark it surfaced **time-rotted oracles**: baseline tests failing in the raw image because the

calendar passed dates baked into the instance. A benchmark’s instances age even as its freshness defeats contamination.

5.3 The recovery-policy contrast, unblinded with its pre-committed analysis

Two more arms ran the same 40 instances under the event-producing stack: **repair-in-place** and **no recovery** (the failed step’s unverified tree is kept). The paired analysis — exact two-sided sign tests on final-state contamination (primary) and onset exposure (co-primary) — was committed before either arm finished.

arm	resolve	cells with final-state contamination	spend
rollback	13/35 = 37%	1	\$8.14
repair-in-place	24/40 = 60%	5	\$14.73
no recovery	22/40 = 55%	9	\$4.40

Primary: no recovery leaves more contamination in the final state than rollback — worse on 9 paired instances, better on 1, ties 25; exact sign $p = 0.022$. Repair-in-place sits between (worse on 5, better on 1; $p = 0.22$). Exposure does not differ (co-primary $p = 0.45$): the damage happens under every policy; the policy decides what *persists*. One no-recovery tree ships with \$(cat django/db/models/expressions.py) as line 1 of the module — a shell idiom pasted into a file write — and every one of its 137 graded tests missing; under rollback the same model’s failed attempt on the same instance was reset and the run graded 137/137.

The tradeoff, reported because it cuts against the harness’s own founding thesis: **rollback resolves fewer tasks**. Against repair-in-place it wins 1 discordant pair and loses 9 (McNemar exact $p = 0.022$); against no recovery 3 and 9 ($p = 0.15$). The verifier gating the rollback is a planner-written check, and it rejects work the official grader accepts; rollback then destroys good patches on false rejections. Recovery policy trades resolution for cleanliness, and the exchange rate is set by verifier precision — a quantity no leaderboard reports.

One disclosure. The first unblinding reported the primary at $p = 0.375$. Seven cells — five no-recovery, two repair — had been dropped as "ungradable" because their final trees made the test suite uncollectable and the grader returned no verdict. Under official semantics that is every test failed, and those were the most catastrophic outcomes in the sweep, vanishing from the endpoint they were the strongest evidence for. The grader now disambiguates (a baseline run proves the environment alive) and the archived trees were re-graded with no model calls. Defect count at submission: twenty-eight in the census, and this one after it.

6 What this means for an agent OS

What to steal

1. **Type your silence** (§4.2, rule one) — in the schema, not in convention.
2. **Prove the instrument alive, not just non-lying** (§4.2, rule two), with positive controls that include *recovered* events.
3. **Never consume output without asserting the producer succeeded** (§4.3) — including your own ledger, fixtures, and event stream.
4. **First contact with a clean host is a validation step**. Six of our defects could not exist on the machine that wrote them.
5. **One environment**. The agent executes where it is measured, enforced by architecture. Our instrument had this seam for its probes and not for its agent; that asymmetry cost the most.

The taxonomy is the argument these rules generalise: the mechanisms are properties of subprocess-and-PATH, containers, parsers, and git — not of our code. Class A is OpenHands’ issues #4235/#7044 [6]; class D is the SWE-bench grader consuming forgeable stdout [5]; the oracle insufficiency UTBoost documents at leaderboard scale [4] is class B one level up, and the time-rotted oracles of §5.2 are its temporal form. And §5.1’s measurements give the rules a stake: the regressions that matter were *created and erased between* the states existing evaluation looks at. An OS layer that standardises recovery without standardising timeline observability will make agent runs look cleaner while hiding exactly this.

7 Related work

SWE-bench [1] and Verified [2] are the substrate. TDAD [3] measures regressions from the final patch — the design §2 argues undercounts recovered events by construction; §5.1 measures the undercount directly: 184 events, 1 final-state-visible, across every bearing run in our data. UTBoost [4] audits the oracle itself. The grader’s stdout-trust defect [5] and OpenHands’ environment issues [6] are our classes D and A in the wild. SWE-bench-Live [7] addresses contamination with rolling instances (median held-out oracle $\sim 34\times$ Verified’s) and exhibits the aging-oracle failure §5.2 reports. Broader evaluation-infrastructure work [8] catalogues agent-eval irreproducibility; our contribution is mechanistic: what breaks inside one instrument, which invariants stop each class, and the first event-level regression measurements on this benchmark.

8 Limitations

Everything is exploratory and dev-slice: the contrast is one sweep per arm on the development slice, and the confirmatory held-out frame is registered but not run. Each cross-stack comparison is a single sweep per stack — sampling variability across re-runs of the same regime is unmeasured until the seeded replications in the confirmatory plan. The regime boundary includes the provider adapter — the two stacks traverse different wire paths, which near-par resolve rates bound but cannot eliminate. The oracle observes roughly the gold test-patch’s blast radius, so "regression" here means regression in the tests adjacent to the edit; the observation hook fails open, so a missed observation thins the timeline symmetrically across stacks (its error counter is reported per cell). Binomial CIs treat the fixed dev slice as the population and ignore repository clustering (46% of the frame is one repo). The catalogue is a census of one team’s instrument; its external instances are evidence of reach, not a survey.

References

- [1] Jimenez et al. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR 2024. arXiv:2310.06770.
- [2] OpenAI. *Introducing SWE-bench Verified*. 2024.
- [3] TDAD. arXiv:2603.17973.
- [4] Yu et al. *UTBoost: Rigorous Evaluation of Coding Agents on SWE-Bench*. ACL 2025. arXiv:2506.09289.
- [5] SWE-bench issue #601: *Test Result Hijacking via Stdout Forging in Evaluation Harness*.
- [6] OpenHands issues #4235, #7044: environment errors in SWE-bench instance evaluation.
- [7] Zhang et al. *SWE-bench Goes Live!* arXiv:2505.23419.
- [8] *Holistic Agent Leaderboard*. arXiv:2510.11977.